

Cheat Sheet Réseau - BGP & Bases

Cheat Sheet Réseau



Ce document répond aux questions essentielles sur les réseaux : GNS3, BGP, et les couches OSI.

1 Le fonctionnement de GNS3

Qu'est-ce que GNS3 ?

GNS3 (Graphical Network Simulator 3) est un **simulateur de réseau open-source** qui permet de créer, configurer et tester des topologies réseau complexes sans avoir besoin de matériel physique.

Comment ça fonctionne ?

Principe de base :

- GNS3 émule des équipements réseau réels (routeurs, switches, firewalls)
- Il utilise des images IOS réelles (Cisco, Juniper, etc.) ou des machines virtuelles
- Vous créez des topologies en glissant-déposant des appareils et en les connectant

Architecture :

- **Client GNS3** : Interface graphique où vous dessinez votre réseau
- **Serveur GNS3** : Moteur qui exécute les simulations (local ou distant)
- **Hyperviseur** : VMware, VirtualBox ou QEMU pour virtualiser les équipements

Exemples d'utilisation :

Cas pratique 1 : Tester une configuration BGP

- Créer 3 routeurs virtuels
- Les interconnecter
- Configurer BGP sans risquer de casser un vrai réseau

Cas pratique 2 : Préparation certification

- Simuler des scénarios d'examens (CCNA, CCNP)
- Tester des configuration ip addr add 30.1.1.1/24 dev eth1s avant déploiement en production

Avantages :

-  Économique (pas besoin d'acheter du matériel)
-  Flexibilité totale pour expérimenter
-  Possibilité de sauvegarder et partager des topologies
-  Intégration avec du matériel réel possible

2 Le fonctionnement global et l'intérêt de BGP

Qu'est-ce que BGP ?

BGP (Border Gateway Protocol) est le **protocole de routage utilisé sur Internet**. C'est le protocole qui permet aux différents réseaux autonomes (AS) de communiquer entre eux.

Fonctionnement global :

Concept de base :

- BGP est un protocole de routage **externe** (contrairement à OSPF ou EIGRP qui sont internes)
- Il échange des informations de routage entre **Autonomous Systems (AS)**
- Chaque AS a un numéro unique (ASN) attribué par les autorités régionales

Comment ça marche ?

▼ Étape 1 : Établissement de session

- Deux routeurs BGP deviennent "voisins" (neighbors)
- Connexion TCP sur le port 179
- Échange de messages OPEN, UPDATE, KEEPALIVE, NOTIFICATION

▼ Étape 2 : Échange de routes

- Chaque routeur annonce les réseaux qu'il connaît

- Les routes incluent des **attributs** (AS-PATH, NEXT-HOP, LOCAL-PREF, MED)
- Le meilleur chemin est sélectionné selon des critères précis

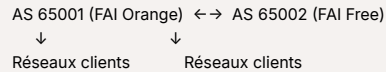
▼ Étape 3 : Propagation

- Les routes apprises sont propagées aux autres voisins BGP
- Mécanisme anti-boucle via l'AS-PATH

Types de BGP :

- **eBGP (external)** : Entre différents AS → routage Internet
- **iBGP (internal)** : À l'intérieur d'un AS → cohérence interne

Exemple concret :



Quand vous accédez à un site web :

1. Votre FAI (AS 65001) utilise BGP pour savoir comment atteindre le réseau du serveur
2. BGP trouve le meilleur chemin parmi plusieurs AS
3. Les paquets traversent plusieurs AS jusqu'à destination

Intérêt de BGP :

Pourquoi l'utiliser ?

- ✓ **Scalabilité** : Gère des millions de routes (toute la table Internet)
- ✓ **Contrôle** : Politiques de routage très fines (traffic engineering)
- ✓ **Redondance** : Multi-homing (plusieurs connexions vers Internet)
- ✓ **Stabilité** : Évite les boucles de routage à l'échelle mondiale

Cas d'usage :

- Entreprise avec plusieurs FAI (redondance)
- Datacenter interconnectant plusieurs sites
- FAI échangeant du trafic avec d'autres FAI

Attributs BGP importants :

Attribut	Description	Exemple d'utilisation
AS-PATH	Liste des AS traversés	Détection de boucles, préférence de chemin
NEXT-HOP	Prochain saut pour atteindre la destination	Résolution du routage
LOCAL-PREF	Préférence locale (iBGP)	Choisir la sortie préférée
MED	Métrique externe	Influencer le trafic entrant

3 Les différences entre la couche 2 et 3 d'un réseau

Vue d'ensemble du modèle OSI :

Le modèle OSI définit 7 couches. Les couches 2 et 3 sont fondamentales pour le routage et la commutation.

Couche 2 : Liaison de données (Data Link)

Rôle :

- Assure la **communication entre équipements directement connectés** sur le même segment réseau
- Détection et correction d'erreurs
- Contrôle de flux

Protocoles :

- Ethernet
- Wi-Fi (802.11)
- PPP
- VLAN (802.1Q)

Adressage : MAC (Media Access Control)

- Format : **AA:BB:CC:DD:EE:FF** (6 octets, 48 bits)
- Adresse physique gravée dans la carte réseau
- Portée : **locale** (ne traverse pas les routeurs)

Équipements :

- **Switch** (commutateur)
- **Bridge** (pont)
- **Access Point Wi-Fi**

Exemple concret :

```
PC1 [MAC: AA:AA:AA:AA:AA:01]
↓
Switch (apprend les MAC, forward les trames)
↓
PC2 [MAC: AA:AA:AA:AA:AA:02]
```

Le switch utilise une **table MAC** pour savoir sur quel port envoyer la trame.

Couche 3 : Réseau (Network)

Rôle :

- Assure le **routage entre différents réseaux**
- Fragmentation et réassemblage des paquets
- Gestion du chemin optimal (routage)

Protocoles :

- IP (IPv4, IPv6)
- ICMP (ping)
- OSPF, BGP, EIGRP (protocoles de routage)

Adressage : IP

- Format IPv4 : 192.168.1.10 (4 octets, 32 bits)
- Format IPv6 : 2001:0db8::1 (16 octets, 128 bits)
- Adresse logique configurable
- Portée : **globale** (routage Internet)

Équipements :

- **Routeur**
- **Firewall L3**
- **Switch L3** (switch avec fonction routage)

Exemple concret :

```
Réseau A: 192.168.1.0/24
↓
Routeur (prend décision de routage basée sur IP)
↓
Réseau B: 192.168.2.0/24
```

Le routeur utilise une **table de routage** pour déterminer le chemin vers la destination.

Tableau comparatif détaillé :

Critère	Couche 2	Couche 3
Nom	Liaison de données	Réseau
Unité de données	Trame (Frame)	Paquet (Packet)
Adressage	Adresse MAC (physique)	Adresse IP (logique)
Portée	Locale (segment réseau)	Globale (inter-réseaux)
Équipement principal	Switch	Routeur
Fonction principale	Commutation	Routage
Exemple d'adresse	AA:BB:CC:DD:EE:FF	192.168.1.1
Broadcast	FF:FF:FF:FF:FF:FF	255.255.255.255

Exemple complet : Communication entre deux PC sur des réseaux différents



Scénario : PC1 (192.168.1.10) veut envoyer un paquet à PC2 (192.168.2.20)

Étapes :

1. **Couche 3** : PC1 voit que PC2 n'est pas sur le même réseau → envoie vers la passerelle (routeur)
 2. **Couche 2** : PC1 encapsule le paquet IP dans une trame Ethernet avec :
 - MAC source : MAC de PC1
 - MAC destination : MAC du routeur
 3. **Switch** (Couche 2) : Forward la trame vers le routeur
 4. **Routeur** (Couche 3) :
 - Décapsule la trame
 - Regarde l'IP destination (192.168.2.20)
 - Consulte sa table de routage
 - Détermine l'interface de sortie
 5. **Routeur** (Couche 2) : Réencapsule dans une nouvelle trame avec :
 - MAC source : MAC du routeur
 - MAC destination : MAC de PC2
 6. **PC2** reçoit la trame et traite le paquet
- Note importante** : Les adresses IP restent identiques tout au long du trajet, mais les adresses MAC changent à chaque saut (routeur).

4 Logiciel de routage de paquet

Qu'est-ce qu'un logiciel de routage de paquet ?

Un **logiciel de routage de paquet** (ou routing software) est un programme qui implémente les algorithmes et protocoles de routage pour diriger les paquets IP d'un réseau à un autre.

Fonctionnement :

Rôle principal :

- Calculer les meilleures routes vers les destinations
- Maintenir et mettre à jour la table de routage du système
- Gérer les protocoles de routage (BGP, OSPF, RIP, etc.)
- Transmettre les paquets selon les règles définies

Architecture typique :

- **Plan de contrôle** : Décide où envoyer les paquets (calcul des routes)
- **Plan de données** : Transfère réellement les paquets (forwarding)

Exemples de logiciels de routage :

FRRouting (FRR) :

- Suite open-source complète
- Supporte BGP, OSPF, IS-IS, RIP, EIGRP, PIM
- Utilisé dans les environnements Linux

Quagga :

- Ancêtre de FRR (maintenant obsolète)
- Remplacé par FRRouting

BIRD :

- Daemon de routage léger et performant
- Très utilisé dans les IXP (Internet Exchange Points)

Cisco IOS / Junos :

- Systèmes d'exploitation propriétaires des routeurs Cisco et Juniper

Exemple d'utilisation :

```
# Sur un serveur Linux avec FRRouting installé
sudo systemctl start frr
sudo vtysh # Interface de configuration style Cisco
```

Un routeur Linux peut ainsi gérer du trafic réseau comme un routeur matériel professionnel.

5 Le service bgpd

Qu'est-ce que bgpd ?

bgpd est un **daemon** (service en arrière-plan) qui implémente spécifiquement le protocole BGP sur un système Linux ou Unix.

Rôle de bgpd :

Fonctions principales :

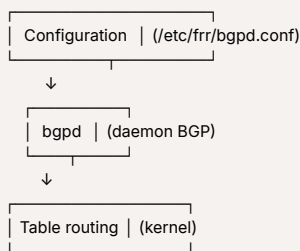
- Établir et maintenir les sessions BGP avec les voisins (peers)
- Échanger les informations de routage BGP
- Calculer les meilleurs chemins selon les politiques définies
- Injecter les routes BGP dans la table de routage du système

Fait partie de :

- FRRouting (FRR)
- Quagga (ancienne version)

Comment ça fonctionne ?

Architecture :



Exemple de configuration :

```
# Démarrer le service bgpd
sudo systemctl start bgpd

# Accéder à la CLI
sudo vtysh

# Configuration BGP
router bgp 65001
neighbor 10.0.0.2 remote-as 65002
network 192.168.1.0/24
```

Cas d'usage concret :

Une entreprise qui veut annoncer son propre bloc d'adresses IP sur Internet :

- Configure bgpd sur un routeur Linux
- Établit des sessions BGP avec ses FAI
- Annonce ses préfixes IP via BGP
- Reçoit les routes Internet de ses peers

6 Le service ospfd

Qu'est-ce que ospfd ?

ospfd est un **daemon** qui implémente le protocole OSPF (Open Shortest Path First) sur un système Linux ou Unix.

Qu'est-ce qu'OSPF ?

OSPF est un protocole de routage **interne** (IGP - Interior Gateway Protocol) qui utilise l'algorithme du plus court chemin (Dijkstra).

Différence avec BGP :

- **OSPF** : À l'intérieur d'un AS (réseau d'entreprise)
- **BGP** : Entre différents AS (Internet)

Rôle de ospfd :

Fonctions principales :

- Découvrir automatiquement les routeurs voisins

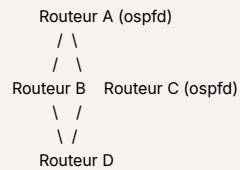
- Échanger les informations sur l'état des liens (LSA - Link State Advertisements)
- Calculer les chemins les plus courts vers toutes les destinations
- Mettre à jour dynamiquement la table de routage

Fonctionnement OSPF :

Étapes :

1. **Découverte** : Les routeurs s'envoient des messages HELLO
2. **Adjacence** : Formation de relations entre voisins
3. **Échange** : Partage des informations de topologie (LSA)
4. **Calcul** : Chaque routeur calcule le plus court chemin (algorithme SPF)
5. **Convergence** : Tous les routeurs ont la même vision du réseau

Exemple de topologie OSPF :



Si un lien tombe, OSPF recalcule automatiquement les routes en quelques secondes.

Exemple de configuration :

```

# Démarrer ospfd
sudo systemctl start ospfd

# Configuration
router ospf
 network 192.168.1.0/24 area 0
 network 10.0.0.0/8 area 0
  
```

Cas d'usage :

Réseau d'entreprise multi-sites :

- Campus avec plusieurs bâtiments
- Chaque bâtiment a un routeur
- OSPF permet de router automatiquement entre tous les bâtiments
- Si un lien tombe, le trafic est redirigé automatiquement

7 Le moteur de routage

Qu'est-ce qu'un moteur de routage ?

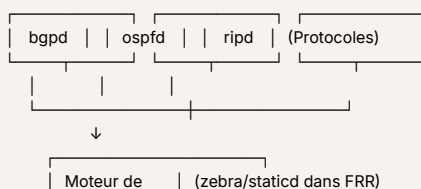
Le **moteur de routage** (routing engine) est le composant central qui orchestre tous les processus de routage sur un routeur.

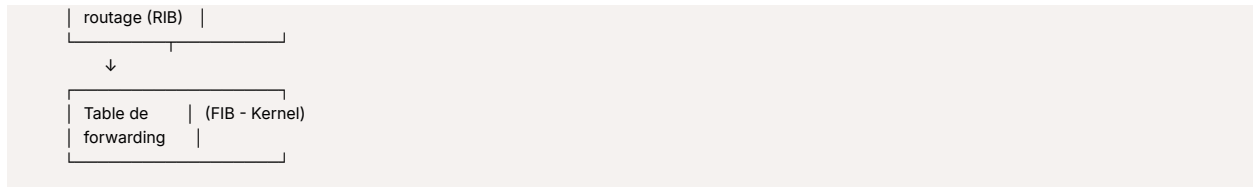
Rôle du moteur de routage :

Fonctions principales :

- Centraliser les informations de tous les protocoles de routage
- Arbitrer entre différentes sources de routes
- Maintenir la RIB (Routing Information Base)
- Pousser les meilleures routes vers la FIB (Forwarding Information Base)
- Gérer la communication entre les daemons de routage

Architecture simplifiée :





Dans FRRouting : zebra

zebra est le moteur de routage de FRR :

- Reçoit les routes de bgpd, ospfd, etc.
- Compare les routes selon les distances administratives
- Installe les meilleures routes dans le kernel Linux

Distance administrative (priorité) :

Source	Distance
Route connectée	0 (priorité max)
Route statique	1
OSPF	110
BGP	200

Exemple concret :

Si ospfd et bgpd annoncent tous les deux une route vers `8.8.8.8` :

- Le moteur de routage choisit OSPF (distance 110 < 200)
- Il installe la route OSPF dans la table de forwarding
- Les paquets suivront le chemin OSPF

8 BusyBox

Qu'est-ce que BusyBox ?

BusyBox est un logiciel qui combine des versions simplifiées de nombreux utilitaires Unix courants dans un **seul exécutable léger**.

Concept :

Slogan : "The Swiss Army Knife of Embedded Linux"

Au lieu d'avoir des centaines de petits programmes séparés (`ls`, `cat`, `grep`, `ping`, etc.), BusyBox regroupe tout en un seul fichier binaire.

Pourquoi l'utiliser ?

Avantages :

- **✓ Léger** : 1-2 MB au lieu de 50+ MB pour les outils standards
- **✓ Économie mémoire** : Un seul binaire en mémoire
- **✓ Simplicité** : Facile à déployer
- **✓ Complet** : Plus de 300 commandes disponibles

Inconvénients :

- **⚠ Versions simplifiées** (moins d'options que les outils GNU)
- **⚠ Moins de fonctionnalités avancées**

Commandes réseau dans BusyBox :

BusyBox inclut des outils réseau essentiels :

- `ping`, `traceroute`
- `ifconfig`, `ip`
- `route`
- `wget`, `telnet`
- `nc` (netcat)

Exemple d'utilisation :

```
# Toutes ces commandes sont en réalité le même exécutable
/bin/busybox ls
/bin/busybox ping 8.8.8.8
/bin/busybox ifconfig eth0
```

```
# Liens symboliques
ls → busybox
ping → busybox
ifconfig → busybox
```

Cas d'usage typiques :

1. Systèmes embarqués :

- Routeurs domestiques (TP-Link, Netgear, etc.)
- IoT devices
- Caméras IP

2. Conteneurs Docker :

- Images Alpine Linux (très légères) utilisent BusyBox
- Images de base minimales

3. Systèmes de récupération :

- LiveCD/USB de dépannage
- Initramfs (système de démarrage)

Exemple : Routeur virtuel minimaliste



Dans un projet réseau, vous pourriez créer un routeur ultra-léger :

- Image Alpine Linux (5 MB) avec BusyBox
-
- FRRouting pour le routage
- = Routeur complet en moins de 50 MB

Comparaison :

Distribution standard (Debian) : ~150 MB minimum
Alpine Linux + BusyBox : ~5 MB

Gain d'espace : 97% de réduction

9 VXLAN et différences avec VLAN

Qu'est-ce qu'un VLAN ?

VLAN (Virtual Local Area Network) est une technologie de **segmentation réseau au niveau de la couche 2** qui permet de créer des réseaux locaux virtuels distincts sur le même équipement physique.

Principe :

- Un VLAN sépare logiquement les équipements sur un même switch
- Identifié par un numéro (VLAN ID) entre 1 et 4094
- Norme IEEE 802.1Q
- Limite : **4096 VLANs maximum**

Qu'est-ce qu'un VXLAN ?

VXLAN (Virtual Extensible LAN) est une technologie de **virtualisation réseau qui encapsule les trames Ethernet dans des paquets UDP/IP** pour créer des réseaux overlay.

Principe :

- Extension du concept VLAN pour les datacenters modernes
- Identifié par un VNI (VXLAN Network Identifier) sur 24 bits
- Limite : **16 millions de segments** (2^{24})
- Encapsulation de trame L2 dans un paquet L3

Différences VLAN vs VXLAN :

Critère	VLAN	VXLAN
Couche OSI	Couche 2 (Liaison)	Couche 2 encapsulé dans Couche 3
Nombre max	4096 VLANs	16 millions de VNI
Portée	Locale (même datacenter)	Globale (peut traverser Internet)

Critère	VLAN	VXLAN
Transport	Trame Ethernet native	UDP/IP (port 4789)
Identifiant	VLAN ID (12 bits)	VNI (24 bits)
Isolation	Au niveau du switch	Au niveau réseau (tunnel)
Use case	Réseau d'entreprise classique	Cloud, datacenter multi-tenant

Comment fonctionne VXLAN ?

Encapsulation :

```

Trame Ethernet originale
↓
[Ethernet | IP | UDP | VXLAN | Ethernet originale | Données]
↓
Transport sur réseau IP

```

Composants VXLAN :

- **VTEP** (VXLAN Tunnel Endpoint) : Point d'entrée/sortie du tunnel
- **VNI** : Identifiant du segment réseau virtuel
- **Underlay** : Réseau physique IP qui transporte VXLAN
- **Overlay** : Réseau virtuel créé par VXLAN

Exemple concret VLAN :

```

Switch avec 3 VLANs
├─ VLAN 10 : Comptabilité (10.0.10.0/24)
├─ VLAN 20 : Marketing (10.0.20.0/24)
└─ VLAN 30 : IT (10.0.30.0/24)

```

Isolation : Un PC du VLAN 10 ne peut pas communiquer directement avec un PC du VLAN 20

Exemple concret VXLAN :



Datacenter multi-sites avec VXLAN

- Datacenter Paris : VTEP 1 (VNI 1000 pour les serveurs web)
- Datacenter Lyon : VTEP 2 (VNI 1000 pour les serveurs web)
- Les serveurs web des deux datacenters sont sur le même réseau L2 virtuel
- Ils peuvent communiquer comme s'ils étaient sur le même switch physique

Cas d'usage :

VLAN :

- Segmentation réseau dans une entreprise
- Séparation départements (RH, IT, Finance)
- Limitation du broadcast

VXLAN :

- Cloud computing (AWS, Azure, OpenStack)
- Migration de VMs entre datacenters
- Multi-tenant isolation dans les datacenters
- Dépassement de la limite des 4096 VLANs

10 Le Switch

Qu'est-ce qu'un switch ?

Un **switch** (commutateur) est un équipement réseau de **couche 2** qui connecte plusieurs appareils sur un réseau local et **forward les trames Ethernet** de manière intelligente.

Fonctionnement d'un switch :

Principe de base :

1. **Apprentissage** : Le switch apprend les adresses MAC en observant les trames
2. **Forwarding** : Il envoie les trames uniquement vers le port destination

3. **Filtrage** : Il ne propage pas les trames inutilement

Table MAC (CAM Table) :

Adresse MAC	Port
AA:AA:AA:AA:AA:01	Port 1
BB:BB:BB:BB:BB:02	Port 2
CC:CC:CC:CC:CC:03	Port 3

Processus de forwarding :

▼ **Scénario : PC1 envoie une trame à PC2**

1. Le switch reçoit la trame sur le port 1
2. Il lit l'adresse MAC source et met à jour sa table (PC1 → Port 1)
3. Il lit l'adresse MAC destination (PC2)
4. Il consulte sa table MAC
5. Il forward la trame uniquement vers le port où se trouve PC2

▼ **Cas spécial : MAC destination inconnue**

Si le switch ne connaît pas la MAC destination :

- Il fait du **flooding** : envoie la trame sur tous les ports (sauf celui d'origine)
- Le bon destinataire répondra, et le switch apprendra sa position

Types de switches :

1. Switch non-manageable (unmanaged) :

- Plug-and-play, aucune configuration
- Pas de VLAN, pas de monitoring
- Usage domestique

2. Switch manageable (managed) :

- Configuration via CLI ou interface web
- Support VLANs, QoS, monitoring, SNMP
- Usage professionnel

3. Switch Layer 3 (L3) :

- Fonctions switch + routeur
- Peut router entre VLANs
- Performance élevée

Différence Switch vs Hub :

Critère	Hub	Switch
Intelligence	Aucune	Apprend les MAC
Forwarding	Broadcast sur tous les ports	Unicast vers le port destination
Collisions	Fréquentes	Éliminées
Performance	Faible	Élevée
Couche OSI	Couche 1 (Physique)	Couche 2 (Liaison)

Exemple d'utilisation :

```
# Configuration d'un switch manageable (Cisco)
enable
configure terminal

# Créer un VLAN
vlan 10
name Comptabilite

# Assigner un port au VLAN
interface GigabitEthernet0/1
switchport mode access
switchport access vlan 10
```

Avantages du switch :

- ✓ **Performance** : Communication full-duplex sans collision
- ✓ **Sécurité** : Isolation du trafic (pas de sniffing facile)

- ✅ **Scalabilité** : Support de centaines de ports
- ✅ **VLANs** : Segmentation logique du réseau

1 1 Le Bridge

Qu'est-ce qu'un bridge ?

Un **bridge** (pont réseau) est un équipement de **couche 2** qui **connecte deux segments réseau** et forward les trames entre eux de manière sélective.

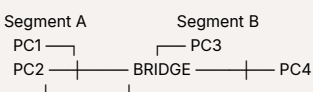
Fonction principale :

Rôle :

- Relier deux réseaux locaux (LAN)
- Filtrer le trafic entre les segments
- Réduire les collisions en segmentant le domaine de collision
- Étendre la portée d'un réseau

Fonctionnement d'un bridge :

Principe :



▼ Processus de décision :

1. Le bridge apprend les adresses MAC de chaque segment
2. Si PC1 envoie à PC2 (même segment) → Le bridge **ne forward pas**
3. Si PC1 envoie à PC3 (autre segment) → Le bridge **forward la trame**

Table de filtrage :

Le bridge maintient une table indiquant sur quel segment se trouve chaque MAC :

Adresse MAC	Segment
PC1 MAC	Segment A
PC2 MAC	Segment A
PC3 MAC	Segment B
PC4 MAC	Segment B

Différence Bridge vs Switch :

Historiquement :

- **Bridge** : Peu de ports (2-4), logiciel, plus lent
- **Switch** : Beaucoup de ports (8-48+), matériel (ASIC), très rapide

Aujourd'hui :

- Un switch est essentiellement un **bridge multi-ports**
- Le terme "bridge" est moins utilisé dans le matériel moderne



En résumé : Un switch est un bridge évolué avec plus de ports et de meilleures performances. Le principe de fonctionnement reste le même (apprentissage MAC + forwarding sélectif).

Bridge logiciel (Linux) :

Sous Linux, vous pouvez créer un bridge logiciel pour connecter des interfaces réseau :

```

# Créer un bridge
ip link add br0 type bridge

# Ajouter des interfaces au bridge
ip link set eth0 master br0
ip link set eth1 master br0

# Activer le bridge
ip link set br0 up
  
```

Cas d'usage du bridge logiciel :

- Virtualisation (connecter des VMs)
- Conteneurs Docker
- Routeurs Linux personnalisés

Types de bridges :

1. Transparent bridge :

- Invisible pour les équipements du réseau
- Apprend automatiquement (comme décrit ci-dessus)

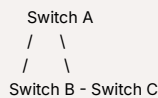
2. Source-routing bridge :

- L'émetteur spécifie le chemin complet
- Rarement utilisé aujourd'hui

Spanning Tree Protocol (STP) :

Quand plusieurs bridges/switches sont interconnectés, il y a un risque de **boucles réseau**.

Problème sans STP :



Une trame peut tourner en boucle indéfiniment → Broadcast storm

Solution : STP (IEEE 802.1D)

- Désactive automatiquement certains liens
- Crée une topologie en arbre sans boucle
- Active les liens de secours si un lien principal tombe

1 2 Différences entre Broadcast et Multicast

Qu'est-ce que le Broadcast ?

Broadcast signifie **diffusion à tous les équipements** d'un réseau.

Principe :

- Un émetteur envoie un paquet à **tous les destinataires possibles**
- Utilisé quand on ne connaît pas l'adresse du destinataire
- Tous les équipements du segment reçoivent et traitent le paquet

Adresses broadcast :

- **Couche 2 (MAC) :** `FF:FF:FF:FF:FF:FF`
- **Couche 3 (IPv4) :** `255.255.255.255` (broadcast général) ou `192.168.1.255` (broadcast réseau)

Qu'est-ce que le Multicast ?

Multicast signifie **diffusion à un groupe spécifique** d'équipements intéressés.

Principe :

- Un émetteur envoie un paquet à **un groupe d'abonnés**
- Seuls les équipements qui ont rejoint le groupe multicast reçoivent
- Plus efficace que le broadcast pour les communications de groupe

Adresses multicast :

- **IPv4 :** `224.0.0.0` à `239.255.255.255`
- **IPv6 :** `FF00::/8`
- **MAC :** `01:00:5E:xxxx:xx:xx` (pour IPv4 multicast)

Différences Broadcast vs Multicast :

Critère	Broadcast	Multicast
Destinataires	Tous les équipements	Groupe spécifique d'abonnés
Efficacité	Faible (trafic inutile)	Élevée (ciblé)
Portée	Limitée au segment local	Peut traverser les routeurs
Abonnement	Aucun (reçu automatiquement)	Nécessite de rejoindre le groupe
Adresse IPv4	255.255.255.255	224.0.0.0 - 239.255.255.255
Protocole IP	IPv4 uniquement	IPv4 et IPv6

Critère	Broadcast	Multicast
Use case	ARP, DHCP	Streaming vidéo, IPTV

Unicast vs Broadcast vs Multicast :



Analogie simple :

Unicast = Téléphone (1 vers 1) : Appeler une personne spécifique

Broadcast = Haut-parleur dans une gare (1 vers tous) : Annonce à tout le monde

Multicast = Liste de diffusion email (1 vers groupe) : Email aux abonnés d'une liste

Exemples concrets :

Broadcast :

```
# ARP Request (Qui a l'IP 192.168.1.10 ?)
Source: 192.168.1.5
Destination: 255.255.255.255 (broadcast)
→ Tous les équipements reçoivent et vérifient
→ Seul 192.168.1.10 répond
```

DHCP Discovery :

- Un PC démarre sans IP
- Envoie un DHCP Discovery en broadcast
- Tous les serveurs DHCP reçoivent et peuvent répondre

Multicast :

```
# Streaming vidéo IPTV
Serveur → Groupe multicast 239.1.1.1
↓
Abonnés qui ont rejoint le groupe (IGMP)
- TV salon : Reçoit
- TV chambre : Reçoit
- PC bureau : Ne reçoit pas (pas abonné)
```

Protocole OSPF :

- Les routeurs OSPF communiquent via multicast
- Adresse `224.0.0.5` (tous les routeurs OSPF)
- Adresse `224.0.0.6` (routeurs OSPF désignés)

Gestion du multicast : IGMP

IGMP (Internet Group Management Protocol) permet aux équipements de :

- **Rejoindre** un groupe multicast (IGMP Join)
- **Quitter** un groupe multicast (IGMP Leave)
- Aux routeurs de savoir quels groupes sont actifs

```
# Exemple : Rejoindre un groupe multicast sur Linux
ip maddr add 239.1.1.1 dev eth0
```

Avantages et inconvénients :

Broadcast :

- Simple, pas de configuration
- Utile pour la découverte (ARP, DHCP)
- Génère du trafic inutile
- Ne traverse pas les routeurs (limité au LAN)

Multicast :

- Efficace pour les communications de groupe
- Économise la bande passante
- Peut traverser les routeurs (avec routage multicast)
- Plus complexe à configurer
- Nécessite support réseau (IGMP, PIM)

Cas d'usage typiques :

Broadcast :

- Protocoles de découverte (ARP, NDP)
- Attribution d'adresses (DHCP)
- Wake-on-LAN
- Annonces de services (NetBIOS)

Multicast :

- IPTV et streaming vidéo
- Visioconférence multi-participants
- Jeux en ligne
- Protocoles de routage (OSPF, EIGRP)
- Mise à jour logicielle en masse

13 BGP-EVPN (Ethernet VPN)

Qu'est-ce que BGP-EVPN ?

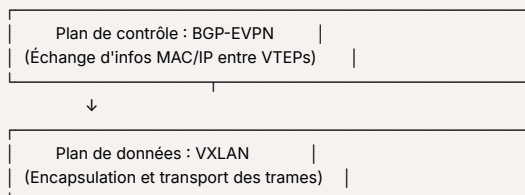
BGP-EVPN (Border Gateway Protocol Ethernet Virtual Private Network) est une **extension de BGP** qui permet de **distribuer les informations de couche 2 (MAC) via BGP** pour créer des réseaux overlay VXLAN à grande échelle.

Concept clé :**Problème sans BGP-EVPN :**

- VXLAN seul utilise du **multicast flood-and-learn** pour apprendre les adresses MAC
- Génère beaucoup de trafic broadcast/multicast inutile
- Difficile à gérer à grande échelle

Solution avec BGP-EVPN :

- Utilise BGP comme **plan de contrôle** pour VXLAN
- Distribution des informations MAC/IP via BGP (plus de flooding)
- Apprentissage centralisé et efficace
- Scalabilité massive pour les datacenters

Architecture BGP-EVPN + VXLAN :**Fonctionnement :****▼ Étape 1 : Un hôte envoie une trame**

1. Le VTEP local reçoit la trame
2. Il apprend l'adresse MAC de l'hôte
3. Il **annonce cette MAC via BGP-EVPN** à tous les autres VTEPs

▼ Étape 2 : Distribution BGP

- Tous les VTEPs reçoivent l'annonce BGP
- Ils mettent à jour leur table MAC
- Plus besoin de flooding

▼ Étape 3 : Communication

- Quand un autre hôte veut communiquer, le VTEP connaît déjà la destination
- Envoi direct via tunnel VXLAN

Avantages de BGP-EVPN :

- ✓ **Efficacité** : Élimine le flooding multicast
- ✓ **Scalabilité** : Peut gérer des dizaines de milliers de VTEPs
- ✓ **Convergence rapide** : Détection rapide des pannes

- ✓ **Multi-tenancy** : Isolation parfaite entre clients
- ✓ **Mobilité** : Facilite la migration de VMs entre sites
- ✓ **Intégration** : S'appuie sur l'infrastructure BGP existante

Cas d'usage :

1. Datacenter moderne (Spine-Leaf) :

- Architecture à grande échelle
- Milliers de serveurs virtualisés
- Multi-tenant cloud

2. DCI (Data Center Interconnect) :

- Connecter plusieurs datacenters
- Étendre les réseaux L2 entre sites
- Migration de workloads

3. Service Provider VPN :

- Offrir des services VPN L2/L3
- Séparation stricte des clients

1 4 Le principe de réflexion de route (Route Reflection)

Qu'est-ce que la Route Reflection ?

La **réflexion de route** est un mécanisme BGP qui résout le problème de scalabilité des sessions iBGP dans les grands réseaux.

Le problème sans Route Reflection

Règle iBGP fondamentale :

- Un routeur iBGP ne propage **pas** les routes apprises d'un pair iBGP vers un autre pair iBGP
- Cette règle évite les boucles de routage

Conséquence :

Pour qu'un réseau fonctionne correctement, **tous les routeurs iBGP doivent être en full-mesh** (connectés entre eux).

Problème de scalabilité :

- 5 routeurs = 10 sessions iBGP
- 10 routeurs = 45 sessions iBGP
- 100 routeurs = 4 950 sessions iBGP

La formule : $n \times (n-1) / 2$ sessions

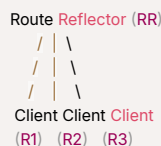


Résultat : Non scalable pour les grands réseaux ! Chaque routeur doit maintenir des dizaines voire des centaines de sessions BGP.

La solution : Route Reflector (RR)

Un **Route Reflector** est un routeur iBGP spécial qui a le droit de **propager les routes iBGP** entre ses clients, brisant ainsi la règle classique d'iBGP.

Architecture :



Rôles dans l'architecture :

- **Route Reflector (RR)** : Le routeur central qui réfléchit les routes
- **Clients** : Les routeurs qui se connectent uniquement au RR
- **Non-clients** : Les autres RR ou routeurs en full-mesh (optionnel)

Fonctionnement détaillé

Règles de réflexion :

▼ 1. Route reçue d'un client

Le RR la **réfléchit** vers :

- ✓ Tous les autres clients

-  Tous les non-clients (autres RR)
-  Tous les pairs eBGP

▼ 2. Route reçue d'un non-client

Le RR la propage vers :

-  Tous les clients seulement
-  Pas vers les autres non-clients (ils sont en full-mesh entre eux)

▼ 3. Route reçue d'un pair eBGP

Le RR la propage vers :

-  Tous les clients
-  Tous les non-clients

Comparaison avant/après

Nombre de routeurs	Sans RR (full-mesh)	Avec 1 RR	Gain
5 routeurs	10 sessions	4 sessions	60%
10 routeurs	45 sessions	9 sessions	80%
50 routeurs	1 225 sessions	49 sessions	96%
100 routeurs	4 950 sessions	99 sessions	98%

Avantages de la Route Reflection

-  **Scalabilité massive** : Réduction drastique du nombre de sessions
-  **Simplicité de gestion** : Configuration centralisée des routes
-  **Économie de ressources** : Moins de CPU/mémoire sur les routeurs clients
-  **Flexibilité** : Possibilité de créer des hiérarchies
-  **Pas de modification des routes** : Les attributs BGP sont préservés

Attributs BGP spécifiques au Route Reflection

ORIGINATOR_ID :

- Identifie le routeur qui a **originellement injecté la route** dans iBGP
- Ajouté par le premier RR qui traite la route
- **Protection anti-boucle** : Un routeur rejette une route dont il est l'ORIGINATOR_ID

CLUSTER_LIST :

- Liste des **clusters** (groupes RR) traversés par la route
- Chaque RR ajoute son CLUSTER_ID à la liste
- **Protection anti-boucle** : Un RR rejette une route contenant déjà son CLUSTER_ID

Attribut	Fonction	Exemple
ORIGINATOR_ID	ID du routeur source	10.0.0.1
CLUSTER_LIST	Liste des clusters traversés	[100, 200, 300]

Exemple de configuration

Sur le Route Reflector (FRRouting) :

```
# Configuration du RR
router bgp 65001
  bgp cluster-id 1.1.1.1

# Définir les clients
neighbor 10.0.0.1 remote-as 65001
neighbor 10.0.0.1 route-reflector-client

neighbor 10.0.0.2 remote-as 65001
neighbor 10.0.0.2 route-reflector-client

neighbor 10.0.0.3 remote-as 65001
neighbor 10.0.0.3 route-reflector-client
```

Sur les clients (routeurs normaux) :

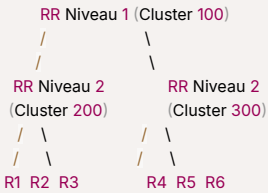
```
# Configuration standard iBGP
router bgp 65001
```



```
neighbor 10.1.1.1 remote-as 65001 # Adresse du RR
# Pas de configuration spéciale côté client
```

Hiérarchie de Route Reflectors

Pour les très grands réseaux, on peut créer une **hiérarchie à plusieurs niveaux** :



Organisation :

- **RR Niveau 1** : Route Reflector principal ("super RR")
- **RR Niveau 2** : Route Reflectors régionaux (clients du RR niveau 1)
- **Routeurs R1-R6** : Clients des RR niveau 2

Chaque cluster a un **CLUSTER_ID** unique pour identifier les groupes.

Exemple de flux de route



Scénario : R1 annonce un nouveau préfixe 192.168.1.0/24

Étape 1 : R1 → RR Niveau 2 (Cluster 200)

- RR ajoute ORIGINATOR_ID = R1
- RR ajoute CLUSTER_LIST = [200]

Étape 2 : RR Niveau 2 → RR Niveau 1

- RR Niveau 1 ajoute à CLUSTER_LIST = [200, 100]

Étape 3 : RR Niveau 1 → RR Niveau 2 (Cluster 300)

- RR Cluster 300 voit que son ID n'est pas dans la liste → accepte
- Ajoute à CLUSTER_LIST = [200, 100, 300]

Étape 4 : RR Cluster 300 → R4, R5, R6

- Les clients reçoivent la route
- ORIGINATOR_ID = R1 (préservé)
- CLUSTER_LIST = [200, 100, 300]

Bonnes pratiques

Redondance des Route Reflectors :

- Déployer **au moins 2 RR** pour la haute disponibilité
- Les clients se connectent aux 2 RR simultanément



Placement optimal :

- RR au cœur du réseau (backbone)
- Bonne connectivité vers tous les clients
- Ressources suffisantes (CPU, mémoire)

Cluster design :

- Limiter le nombre de clients par RR (max 100-200)
- Utiliser des hiérarchies pour les très grands réseaux

- Grouper les clients par région/fonction

Cas d'usage typiques

1. FAI (Fournisseur d'Accès Internet) :

- Des centaines de routeurs PE (Provider Edge)
- 2-4 Route Reflectors centraux
- Propagation efficace des routes client

2. Datacenter :

- Spine switches comme Route Reflectors
- Leaf switches comme clients
- Architecture Clos/Spine-Leaf

3. Entreprise multi-sites :

- RR sur chaque site principal
- Hiérarchie entre les RR des différents sites
- Routeurs de succursales comme clients

Limitations et considérations

⚠ Points d'attention :

- **Point de défaillance** : Le RR est critique (mitiger avec de la redondance)
- **Surcharge** : Le RR doit gérer toutes les routes (dimensionner correctement)
- **Sous-optimalité** : Parfois le chemin choisi n'est pas le meilleur (rare)
- **Complexité** : Plus difficile à déboguer qu'un full-mesh simple

🔧 Solutions :

- Redondance (plusieurs RR)
- Monitoring actif des RR
- Hardware adapté (CPU, RAM suffisants)
- Documentation claire de la topologie

Alternative : BGP Confederations

Une autre solution au problème de scalabilité iBGP :

- Diviser l'AS en **sous-AS** (confederations)
- Chaque sous-AS en full-mesh interne
- eBGP spécial entre sous-AS

Route Reflection vs Confederations :

Critère	Route Reflection	Confederations
Simplicité	✅ Plus simple	❌ Plus complexe
Flexibilité	✅ Très flexible	⚠ Structure rigide
Popularité	✅ Très répandu	⚠ Moins utilisé
Scalabilité	✅ Excellente	✅ Excellente

Recommandation : Route Reflection est généralement préféré pour sa simplicité et sa flexibilité.

15 VTEP et VNI : Composants essentiels de VXLAN

VTEP (VXLAN Tunnel Endpoint)

Définition :

VTEP est l'équipement ou l'interface qui **encapsule et décapsule** les trames Ethernet dans des paquets VXLAN.

Rôle du VTEP :

Sens sortant (Encapsulation) :

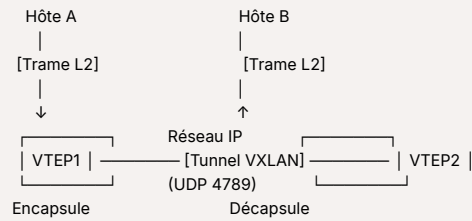
1. Reçoit une trame Ethernet native
2. Ajoute un header VXLAN avec le VNI
3. Encapsule dans UDP/IP
4. Envoie sur le réseau underlay (réseau IP physique)

Sens entrant (Décapsulation) :

1. Reçoit un paquet VXLAN

2. Retire les headers UDP/IP/VXLAN
3. Extrait la trame Ethernet originale
4. Forward vers l'hôte local

Schéma VTEP :



Types de VTEPs :

1. Hardware VTEP :

- Switch physique avec support VXLAN
- Exemple : Top-of-Rack (ToR) switch
- Haute performance (ASIC)

2. Software VTEP :

- Implémenté dans l'hyperviseur
- Exemple : Linux kernel, Open vSwitch, VMware NSX
- Plus flexible, mais consomme du CPU

Identification du VTEP :

Chaque VTEP a :

- Une **adresse IP source** (son adresse sur le réseau underlay)
- Une **adresse IP de destination** (l'autre VTEP du tunnel)
- Un ou plusieurs **VNI**s qu'il gère

Exemple de configuration VTEP (Linux) :

```

# Créer une interface VXLAN
ip link add vxlan10 type vxlan \
  id 10 \
  local 10.0.1.1 \
  remote 10.0.1.2 \
  dstport 4789 \
  dev eth0

# Activer l'interface
ip link set vxlan10 up

# Ajouter au bridge
ip link set vxlan10 master br0

```

Explication :

- **id 10** → VNI = 10
- **local 10.0.1.1** → Adresse IP locale du VTEP
- **remote 10.0.1.2** → Adresse IP du VTEP distant
- **dstport 4789** → Port UDP standard VXLAN

15 VNI (VXLAN Network Identifier)

Définition :

VNI est l'identifiant unique d'un **segment de réseau virtuel** dans VXLAN. C'est l'équivalent du VLAN ID, mais sur 24 bits.

Caractéristiques du VNI :

Format :

- **24 bits** (contrairement aux 12 bits des VLANs)
- Valeurs : 0 à 16 777 215 ($2^{24} - 1$)

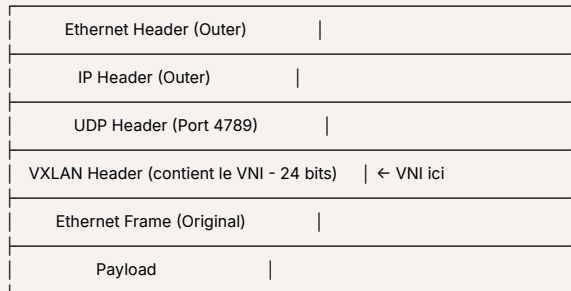
- Permet **16 millions de segments** isolés

Rôle :

- **Isoler** les tenants dans un datacenter multi-tenant
- **Identifier** à quel réseau virtuel appartient une trame
- **Segmenter** le trafic logiquement

Comment le VNI fonctionne :

Encapsulation VXLAN avec VNI :



Exemple concret :



Datacenter multi-tenant :

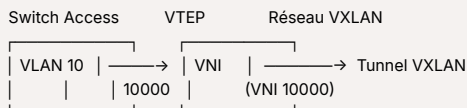
- **VNI 1000** : Réseau du Client A (production)
- **VNI 2000** : Réseau du Client B (production)
- **VNI 1001** : Réseau du Client A (test)
- **VNI 2001** : Réseau du Client B (test)

Chaque VNI est complètement isolé. Les VMs du VNI 1000 ne peuvent pas voir celles du VNI 2000.

Mapping VNI ↔ VLAN :

Dans les architectures hybrides, on peut mapper :

- **VLAN 10** (local) → **VNI 10000** (global)
- **VLAN 20** (local) → **VNI 20000** (global)



VNI dans BGP-EVPN :

Avec BGP-EVPN, le VNI est annoncé via BGP :

- Les VTEPs s'abonnent aux VNIs qui les intéressent
- Apprentissage automatique des MAC pour chaque VNI
- Isolation garantie par le plan de contrôle

16 Routes BGP-EVPN : Types 2 et 3

Qu'est-ce qu'une route EVPN ?

Dans BGP-EVPN, les informations ne sont pas des préfixes IP classiques, mais des **routes EVPN** qui transportent différents types d'informations.

BGP-EVPN définit plusieurs types de routes (appelées NLRI - Network Layer Reachability Information) :

Type	Nom
Type 1	Ethernet Auto-Discovery (EAD)
Type 2	MAC/IP Advertisement
Type 3	Inclusive Multicast Ethernet Tag
Type 4	Ethernet Segment

Type	Nom
Type 5	IP Prefix

Nous allons détailler les **Types 2 et 3**, les plus courants.

Route Type 2 : MAC/IP Advertisement

Rôle :

Annoncer l'**emplacement d'un hôte** identifié par son adresse MAC et/ou IP.

Contenu d'une route Type 2 :

- **MAC Address** : Adresse MAC de l'hôte
- **IP Address** : Adresse IP de l'hôte (optionnelle)
- **VNI** : Identifiant du segment réseau
- **VTEP IP** : Adresse IP du VTEP où se trouve l'hôte
- **Route Target** : Pour le filtrage et la distribution

Fonctionnement Type 2 :

▼ Scénario : Un nouveau serveur se connecte

1. Le serveur envoie un paquet (ex: requête ARP)
2. Le VTEP local apprend la MAC du serveur
3. Le VTEP **crée une route Type 2** avec :
 - MAC = AA:BB:CC:DD:EE:01
 - IP = 10.1.1.10
 - VNI = 1000
 - VTEP = 192.168.1.1
4. Le VTEP **annonce cette route via BGP** à tous les autres VTEPs
5. Tous les VTEPs mettent à jour leur table : "Pour atteindre MAC AA:BB:CC:DD:EE:01, envoyer vers VTEP 192.168.1.1"

Exemple visuel Type 2 :

```

Serveur A (MAC: AA:BB, IP: 10.1.1.10)
↓
VTEP1 (IP: 192.168.1.1)
↓
[Annonce BGP Type 2]
→ MAC: AA:BB
→ IP: 10.1.1.10
→ VNI: 1000
→ VTEP: 192.168.1.1
↓
Tous les autres VTEPs apprennent
  
```

Avantages Type 2 :

- ✓ **Apprentissage proactif** : Pas besoin de flood-and-learn
- ✓ **Optimisation ARP** : Réduction du trafic ARP/ND
- ✓ **Convergence rapide** : Détection immédiate des changements
- ✓ **Mobilité** : Un hôte peut changer de VTEP, la route est mise à jour automatiquement

Route Type 3 : Inclusive Multicast Ethernet Tag (IMET)

Rôle :

Annoncer la **présence d'un VTEP** pour un VNI donné et établir les **tunnels multicast/broadcast**.

Contenu d'une route Type 3 :

- **VNI** : Identifiant du segment réseau
- **VTEP IP** : Adresse IP du VTEP qui s'annonce
- **Route Target** : Pour le filtrage

Fonctionnement Type 3 :

▼ Scénario : Un VTEP rejoint un VNI

1. Un VTEP est configuré pour gérer le VNI 1000
2. Le VTEP **crée une route Type 3** :

- VNI = 1000
 - VTEP = 192.168.1.1
3. Le VTEP **annonce cette route via BGP**
 4. Tous les autres VTEPs du VNI 1000 apprennent l'existence de ce nouveau VTEP
 5. Ils établissent un **tunnel VXLAN** vers ce VTEP

Pourquoi Type 3 est important ?

Gestion du BUM (Broadcast, Unknown unicast, Multicast) :

Quand un VTEP doit envoyer du trafic BUM (ex: broadcast ARP) :

- Il consulte les routes Type 3 reçues
- Il connaît tous les VTEPs membres du VNI
- Il **réplique le paquet** vers chaque VTEP membre
- Plus besoin de multicast sur le réseau underlay

Exemple visuel Type 3 :

```

VTEP1 rejoint VNI 1000
↓
[Annonce BGP Type 3]
→ VNI: 1000
→ VTEP: 192.168.1.1
↓
VTEP2, VTEP3, VTEP4 apprennent
↓
Établissement des tunnels:
VTEP1 ↔ VTEP2
VTEP1 ↔ VTEP3
VTEP1 ↔ VTEP4

```

Avantages Type 3 :

- ✅ **Auto-découverte** : Les VTEPs se découvrent automatiquement
- ✅ **Pas de multicast underlay** : Utilise de l'unicast pour répliquer le BUM
- ✅ **Simplification** : Plus besoin de configurer manuellement les tunnels
- ✅ **Dynamique** : Ajout/retrait de VTEPs sans reconfiguration

Différences Type 2 vs Type 3

Critère	Route Type 2	Route Type 3
Nom	MAC/IP Advertisement	Inclusive Multicast Ethernet Tag
Objet annoncé	Emplacement d'un hôte (MAC/IP)	Présence d'un VTEP
Information clé	MAC, IP, VNI, VTEP	VNI, VTEP
Fréquence	À chaque nouvel hôte détecté	À l'activation d'un VNI sur un VTEP
Usage principal	Apprentissage MAC/IP distribué	Découverte des VTEPs et gestion BUM
Trafic concerné	Unicast (vers un hôte spécifique)	BUM (Broadcast, Unknown, Multicast)
Granularité	Par hôte	Par VTEP

Ordre d'utilisation Type 3 → Type 2 :



Séquence typique de mise en place :

1. Bootstrap (Type 3) :

- Les VTEPs s'annoncent via des routes Type 3
- Établissement des tunnels VXLAN
- Préparation pour le trafic BUM

2. Apprentissage (Type 2) :

- Les hôtes envoient du trafic
- Les VTEPs apprennent les MAC/IP
- Annonce via routes Type 2
- Construction des tables de forwarding

3. Opération stable :

- Trafic unicast optimisé (grâce aux Type 2)
- Trafic BUM géré efficacement (grâce aux Type 3)

Exemple complet dans un datacenter :

Datacenter avec 3 VTEPs et VNI 1000

Étape 1 : Les 3 VTEPs s'annoncent (Type 3)

VTEP1 → BGP Type 3 (VNI 1000, VTEP1)

VTEP2 → BGP Type 3 (VNI 1000, VTEP2)

VTEP3 → BGP Type 3 (VNI 1000, VTEP3)

→ Tous connaissent tous, tunnels établis

Étape 2 : Des VMs démarrent et communiquent (Type 2)

VM-A (MAC: AA:AA, sur VTEP1)

→ VTEP1 annonce Type 2 (MAC AA:AA, VTEP1)

VM-B (MAC: BB:BB, sur VTEP2)

→ VTEP2 annonce Type 2 (MAC BB:BB, VTEP2)

→ Tous savent où joindre VM-A et VM-B

Étape 3 : Communication optimisée

VM-A ping VM-B

→ VTEP1 connaît VTEP2 (grâce à Type 2)

→ Envoi direct via tunnel VXLAN

→ Pas de flooding



Points clés à retenir



GNS3 : Simulateur pour tester des configurations réseau sans matériel physique

BGP : Protocole de routage inter-AS qui fait fonctionner Internet

Couche 2 vs 3 : Commutation locale (MAC) vs Routage global (IP)

bgpd/ospfd : Daemons qui implémentent les protocoles de routage

Moteur de routage : Centralise les décisions de routage (zebra)

BusyBox : Boîte à outils Unix ultra-légère

VLAN vs VXLAN : 4096 VLANs locaux vs 16M VNI sur IP

Switch : Commutateur intelligent L2 avec table MAC

Bridge : Pont entre segments (switch = bridge multi-ports)

Broadcast vs Multicast : Tous vs Groupe d'abonnés

BGP-EVPN : Plan de contrôle BGP pour VXLAN (élimine flooding)

VTEP : Point d'encapsulation/décapsulation VXLAN

VNI : Identifiant de segment réseau (24 bits, 16M possibles)

Route Type 2 : Annonce d'hôte (MAC/IP + VTEP)

Route Type 3 : Annonce de VTEP (auto-découverte + BUM)

Route Reflection : Solution de scalabilité pour iBGP (évite le full-mesh)

Application pratique : Le projet BADASS

Vue d'ensemble du projet

BADASS (BGP At Doors of Autonomous Systems is Simple) est un projet de l'école 42 qui met en pratique **tous les concepts** de ce cheat sheet dans une architecture réseau réelle simulée avec GNS3 et Docker.



Objectif : Construire un mini-datacenter avec VXLAN et BGP-EVPN pour comprendre comment fonctionnent les infrastructures cloud modernes (AWS, Azure, Google Cloud).

Architecture du projet en 3 parties

Le projet BADASS se décompose en 3 parties progressives :

▼ Part 1 : GNS3 configuration avec Docker

Préparation des outils et création des images Docker

▼ Part 2 : Discovering a VXLAN

Création d'un réseau overlay VXLAN basique

▼ Part 3 : Discovering BGP with EVPN

Construction d'un datacenter complet avec BGP-EVPN



Part 1 : Configuration GNS3 avec Docker

Concepts utilisés

Concept du cheat sheet	Application dans P1
GNS3	Installation et configuration du simulateur réseau
BusyBox	Création d'une image Docker légère pour les hosts
bgpd/ospfd/IS-IS	Création d'une image Docker routeur avec FRRouting
Moteur de routage	Configuration de zebra dans l'image routeur

Ce que tu construis dans P1

Image 1 : Host simple

- Basée sur Alpine Linux
- Contient BusyBox pour les commandes réseau de base
- Utilisée pour simuler des machines clientes

Image 2 : Routeur complet

- Basée sur Alpine Linux ou similaire
- Contient FRRouting avec :
 - **zebra** : Moteur de routage central
 - **bgpd** : Daemon BGP
 - **ospfd** : Daemon OSPF
 - **isisd** : Daemon IS-IS
- Utilisée pour tous les équipements réseau (routeurs, switches L3, VTEPs)

Topologie P1

host_vaxcore-1 ↔ routeur_vaxcore
(Image 1) (Image 2)

Objectif : Vérifier que les deux images Docker fonctionnent correctement dans GNS3.

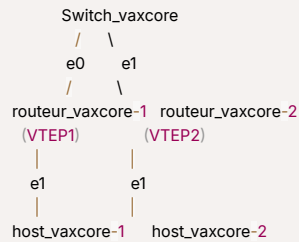
Part 2 : Discovering a VXLAN

Concepts utilisés

Concept du cheat sheet	Application dans P2
VXLAN	Création d'un réseau overlay avec VNI 10
VTEP	Les 2 routeurs agissent comme VTEPs
VNI	Utilisation du VNI 10 pour identifier le segment
Bridge	Création de br0 pour connecter VXLAN et interfaces
Switch	Switch pour interconnecter les 2 routeurs

Concept du cheat sheet	Application dans P2
Multicast	Groupe 239.1.1.1 pour le mode dynamique
Couche 2	Les hosts communiquent en L2 via VXLAN

Topologie P2



Fonctionnement détaillé

Phase 1 : Mode statique

1. Chaque routeur crée une interface VXLAN (vxlan10) avec VNI 10
2. Configuration manuelle du VTEP distant
3. Création d'un bridge (br0) qui relie :
 - L'interface VXLAN
 - L'interface physique vers le host
4. Les deux hosts peuvent communiquer en L2 à travers le tunnel VXLAN

Phase 2 : Mode multicast dynamique

1. Ajout d'un groupe multicast (239.1.1.1) pour la découverte automatique
2. Plus besoin de configurer manuellement le VTEP distant
3. Les VTEPs s'annoncent via multicast
4. Apprentissage automatique des adresses MAC

Exemple de configuration (routeur_vaxcore-1)

```

# Créer l'interface VXLAN en mode multicast
ip link add vxlan10 type vxlan \
  id 10 \
  dstport 4789 \
  group 239.1.1.1 \
  dev eth0

# Créer le bridge
ip link add br0 type bridge

# Ajouter les interfaces au bridge
ip link set vxlan10 master br0
ip link set eth1 master br0

# Activer tout
ip link set vxlan10 up
ip link set br0 up
ip link set eth1 up

```

Ce qui se passe sous le capot

▼ host_vaxcore-1 ping host_vaxcore-2

1. **host_vaxcore-1** envoie un paquet ICMP
2. Le paquet arrive sur **routeur_vaxcore-1** (VTEP1)
3. **VTEP1** encapsule :
 - Trame Ethernet originale
 -
 - Header VXLAN (VNI 10)
 -
 - Header UDP (port 4789)

- Header IP (vers VTEP2)
- Le paquet traverse le réseau underlay (le switch)
 - VTEP2** reçoit et décapsule
 - La trame Ethernet originale arrive sur **host_vaxcore-2**
 - host_vaxcore-2** répond (chemin inverse)



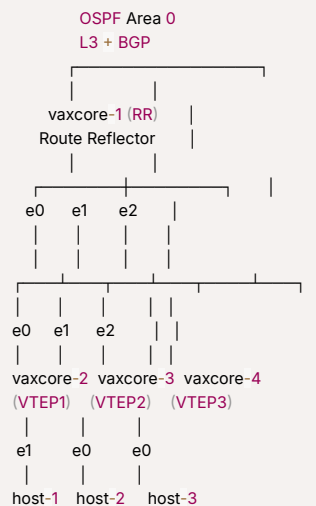
Magie de VXLAN : Les deux hosts pensent être sur le même réseau local (L2), alors qu'ils sont en réalité séparés par un réseau IP (L3) !

Part 3 : Discovering BGP with EVPN

Concepts utilisés (TOUS !)

Concept du cheat sheet	Application dans P3
BGP	Protocole de base pour EVPN
iBGP	Sessions BGP internes entre RR et VTEPs
Route Reflection	RR central évite full-mesh entre VTEPs
OSPF	Routage underlay entre tous les équipements
BGP-EVPN	Plan de contrôle pour distribuer les infos MAC/IP
VXLAN	Plan de données (VNI 10)
VTEP	Les 3 routeurs leafs sont des VTEPs
Route Type 2	Annonce automatique des MAC des hosts
Route Type 3	Annonce de présence des VTEPs
Couche 2 & 3	L3 pour underlay (OSPF), L2 pour overlay (VXLAN)

Topologie P3 : Mini Datacenter



Architecture en couches

Underlay (Couche 3 - Infrastructure physique) :

- **Protocole** : OSPF
- **Rôle** : Assurer la connectivité IP entre tous les équipements
- **AS** : Unique (ex: AS 1) car tout est iBGP

Overlay (Couche 2 - Réseau virtuel) :

- **Protocole** : VXLAN
- **VNI** : 10
- **Rôle** : Créer un réseau L2 virtuel pour les hosts

Plan de contrôle (Intelligence) :

- **Protocole** : BGP-EVPN
- **Rôle** : Distribuer automatiquement les infos MAC/IP entre VTEPs

- **Architecture** : Route Reflection (RR + 3 clients)

Flux de configuration étape par étape

▼ Étape 1 : Configuration OSPF (Underlay)

Sur **tous les routeurs** (RR + VTEPs) :

```
router ospf
network 10.1.1.0/30 area 0
network 1.1.1.0/24 area 0
```

Résultat : Tous les routeurs peuvent se joindre en IP

▼ Étape 2 : Configuration BGP sur le Route Reflector

Sur **vaxcore-1 (RR)** :

```
router bgp 1
bgp router-id 1.1.1.1
bgp cluster-id 1.1.1.1

# Déclarer les 3 VTEPs comme clients
neighbor 1.1.1.2 remote-as 1
neighbor 1.1.1.2 route-reflector-client
neighbor 1.1.1.2 update-source lo

neighbor 1.1.1.3 remote-as 1
neighbor 1.1.1.3 route-reflector-client
neighbor 1.1.1.3 update-source lo

neighbor 1.1.1.4 remote-as 1
neighbor 1.1.1.4 route-reflector-client
neighbor 1.1.1.4 update-source lo

# Activer l'address-family EVPN
address-family l2vpn evpn
neighbor 1.1.1.2 activate
neighbor 1.1.1.3 activate
neighbor 1.1.1.4 activate
exit-address-family
```

▼ Étape 3 : Configuration BGP sur les VTEPs

Sur **vaxcore-2 (VTEP1)** (similaire sur VTEP2 et VTEP3) :

```
router bgp 1
bgp router-id 1.1.1.2

# Session iBGP vers le RR
neighbor 1.1.1.1 remote-as 1
neighbor 1.1.1.1 update-source lo

address-family l2vpn evpn
neighbor 1.1.1.1 activate
advertise-all-vni
exit-address-family
```

▼ Étape 4 : Configuration VXLAN sur les VTEPs

Sur **vaxcore-2 (VTEP1)** :

```
# Créer l'interface VXLAN
ip link add vxlan10 type vxlan \
id 10 \
dstport 4789 \
local 1.1.1.2 \
nolearning

# Créer le bridge
ip link add br0 type bridge
ip link set vxlan10 master br0
ip link set eth1 master br0

# Activer
```

```
ip link set vxlan10 up
ip link set br0 up
```

Note : nolearning car BGP-EVPN gère l'apprentissage

▼ Étape 5 : Les hosts démarrent

Quand **host-1** se connecte :

1. Il envoie une trame (ex: ARP, DHCP)
2. **VTEP1** apprend sa MAC
3. **VTEP1** crée une **route BGP Type 2** :
 - MAC de host-1
 - VNI 10
 - VTEP = 1.1.1.2
4. **VTEP1** envoie cette route au **RR**
5. Le **RR** réfléchit la route vers **VTEP2** et **VTEP3**
6. **VTEP2** et **VTEP3** savent maintenant :
 - "Pour joindre la MAC de host-1, envoyer vers VTEP 1.1.1.2"

Séquence complète : host-1 ping host-2



Avant le ping (Bootstrap) :

1. Routes Type 3 (IMET) :

- VTEP1 annonce : "Je gère VNI 10, je suis 1.1.1.2"
- VTEP2 annonce : "Je gère VNI 10, je suis 1.1.1.3"
- VTEP3 annonce : "Je gère VNI 10, je suis 1.1.1.4"

→ Les 3 VTEPs se connaissent via BGP-EVPN

2. Routes Type 2 (MAC/IP) :

- VTEP1 annonce : "MAC de host-1 = AA:AA:AA:01, chez moi (1.1.1.2)"
- VTEP2 annonce : "MAC de host-2 = BB:BB:BB:02, chez moi (1.1.1.3)"
- VTEP3 annonce : "MAC de host-3 = CC:CC:CC:03, chez moi (1.1.1.4)"

→ Tous les VTEPs connaissent toutes les MACs

Maintenant le ping :

1. **host-1 (10.1.2.1)** veut pinger **host-2 (10.1.2.2)**
→ Envoie un **ARP Request** "Qui a 10.1.2.2 ?"
2. **VTEP1** reçoit l'**ARP Request**
→ Consulte sa table **BGP-EVPN**
→ Sait que **MAC** de **host-2** est chez **VTEP2 (1.1.1.3)**
→ Encapsule dans **VXLAN** vers 1.1.1.3
3. Le paquet traverse le réseau **underlay (OSPF)**
vaxcore-2 → vaxcore-1 → vaxcore-3
4. **VTEP2** reçoit et décapsule
→ Extrait l'**ARP Request** original
→ Forward vers **host-2**
5. **host-2** répond avec un **ARP Reply**
→ **VTEP2** encapsule vers **VTEP1**
→ **VTEP1** décapsule et forward vers **host-1**
6. **host-1** a maintenant la **MAC** de **host-2**
→ Envoie le ping **ICMP**
→ Même processus d'encapsulation/décapsulation
7. **host-2** répond au ping
→ Retour via **VXLAN**

Avantages de cette architecture

✅ **Scalabilité :**

- Grâce au Route Reflector, pas de full-mesh BGP
- 3 sessions iBGP au lieu de 3 (sans RR : 3 sessions aussi, mais avec 10 VTEPs : 9 sessions au lieu de 45)

✅ **Automatisation :**

- Pas besoin de configurer manuellement les tunnels VXLAN
- Apprentissage automatique des MAC via BGP-EVPN
- Pas de flooding inutile

✅ **Flexibilité :**

- Facile d'ajouter un nouveau VTEP (juste une session BGP vers le RR)
- Facile d'ajouter un nouveau host (apprentissage automatique)

✅ **Résilience :**

- Si un VTEP tombe, BGP retire ses routes automatiquement
- OSPF recalcule les chemins underlay si un lien tombe

🔗 **Tableau de correspondance complète**

Section du cheat sheet	Utilisation dans BADASS	Partie du projet
1. GNS3	Plateforme de simulation	P1, P2, P3
2. BGP	Base de BGP-EVPN	P3
3. Couches 2 & 3	L2 overlay + L3 underlay	P2, P3
4. Logiciel de routage	FRRouting dans images Docker	P1
5. bgpd	Daemon BGP pour EVPN	P1, P3
6. ospfd	Routage underlay	P1, P3
7. Moteur de routage	Zebra dans FRRouting	P1, P3
8. BusyBox	Outils réseau dans images hosts	P1, P2, P3
9. VXLAN vs VLAN	VXLAN pour overlay (VNI 10)	P2, P3
10. Switch	Interconnexion dans P2	P2
11. Bridge	br0 pour lier VXLAN et eth	P2, P3
12. Broadcast vs Multicast	Multicast pour VXLAN dynamique	P2
13. BGP-EVPN	Plan de contrôle principal	P3
14. Route Reflection	RR pour éviter full-mesh	P3
15. VTEP et VNI	VTEPs = leafs, VNI = 10	P2, P3
16. Routes Type 2 & 3	Annonces BGP automatiques	P3

🎓 **Pourquoi cette architecture dans le monde réel ?**

L'architecture du projet BADASS (Part 3) est **exactement** celle utilisée dans les datacenters modernes :

AWS / Azure / Google Cloud :

- Utilisent VXLAN pour isoler les réseaux des clients (VPC)
- Utilisent BGP-EVPN pour gérer les millions de VMs
- Utilisent des Route Reflectors pour la scalabilité

Spine-Leaf Architecture :

- **Spines** = Route Reflectors (comme vaxcore-1)
- **Leafs** = VTEPs (comme vaxcore-2, 3, 4)
- **Hosts** = VMs des clients

Avantages pour le cloud :

- **Multi-tenancy** : Chaque client a son VNI (VNI 10 pour client A, VNI 20 pour client B, etc.)
- **Scalabilité** : 16 millions de VNIs possibles vs 4096 VLANs
- **Flexibilité** : Déplacer une VM d'un datacenter à l'autre sans changer son IP
- **Performance** : Pas de flooding, apprentissage proactif via BGP

📝 **Checklist de révision pour BADASS**



Avant de commencer le projet, assure-toi de maîtriser :

Concepts de base :

- ☐ Différence entre couche 2 et couche 3
- ☐ Adressage MAC vs IP
- ☐ Rôle d'un switch vs routeur vs bridge
- ☐ Broadcast vs multicast

VXLAN :

- ☐ Qu'est-ce qu'un VTEP ?
- ☐ Qu'est-ce qu'un VNI ?
- ☐ Comment fonctionne l'encapsulation VXLAN ?
- ☐ Différence entre mode statique et multicast

BGP :

- ☐ Différence entre eBGP et iBGP
- ☐ Qu'est-ce qu'un AS ?
- ☐ Problème du full-mesh iBGP
- ☐ Comment fonctionne la Route Reflection ?

BGP-EVPN :

- ☐ Rôle du plan de contrôle vs plan de données
- ☐ Différence entre routes Type 2 et Type 3
- ☐ Comment les VTEPs se découvrent ?
- ☐ Comment les MACs sont apprises automatiquement ?

OSPF :

- ☐ Rôle d'OSPF dans l'underlay
- ☐ Différence entre OSPF et BGP
- ☐ Qu'est-ce qu'une area OSPF ?

FRRouting :

- ☐ Qu'est-ce que zebra ?
- ☐ Comment configurer bgpd ?
- ☐ Comment configurer ospfd ?
- ☐ Comment vérifier les routes BGP ?



Conseils pour réussir le projet

1. Procède étape par étape

- Ne passe pas à P3 avant d'avoir parfaitement compris P2
- Teste chaque configuration avant d'ajouter de la complexité

2. Utilise les commandes de vérification

Vérifier les routes OSPF
show ip ospf route

Vérifier les sessions BGP
show bgp summary
show bgp l2vpn evpn summary

Vérifier les routes EVPN
show bgp l2vpn evpn
show bgp l2vpn evpn route type 2
show bgp l2vpn evpn route type 3

Vérifier les VTEPs connus
show evpn vni

Vérifier les MACs apprises
show evpn mac vni 10

Vérifier la table de forwarding
bridge fdb show

Vérifier les interfaces VXLAN
ip -d link show vxlan10

3. Documente ta configuration

- Ajoute des commentaires dans tes fichiers de config
- Explique pourquoi tu fais chaque étape
- Ça t'aidera pendant l'évaluation

4. Comprends le "pourquoi"

- Ne te contente pas de copier-coller des commandes
- Comprends ce que fait chaque ligne
- Sois capable d'expliquer chaque concept à l'oral

5. Teste la connectivité

```
# Depuis un host
ping <autre-host>
traceroute <autre-host>

# Capturer le trafic
tcpdump -i any -n icmp
tcpdump -i eth0 -n port 4789 # Voir VXLAN
tcpdump -i eth0 -n proto ospf # Voir OSPF
```

Conclusion

Le projet BADASS est une **mise en pratique complète** de toutes les technologies de networking moderne. En le réalisant, tu comprendras :

- Comment fonctionnent les infrastructures cloud (AWS, Azure, GCP)
- Comment les datacenters gèrent des millions de VMs
- Comment BGP-EVPN a révolutionné le networking L2/L3
- Comment construire des architectures réseau scalables et résilientes



Ce projet te donne les compétences pour :

- Travailler en tant que Network Engineer dans un datacenter
- Comprendre les architectures SDN (Software-Defined Networking)
- Configurer des environnements cloud complexes
- Débuguer des problèmes de connectivité dans des infrastructures modernes

Bon courage pour ton projet BADASS ! 🚀